

Problem statement

Create a C#/Java program to **build an order book and simulate an exchange auction matching algorithm.**

The algorithm should output a matching auction price (see details).

The solution should minimize the use of external libraries. External dependencies should only be used when necessary, such as for mocking in unit tests.

What is “Auction”:

Most financial exchanges have a special trading phase called an **auction**.

During the auction period, market participants can:

- Submit new orders
- Amend existing orders
- Cancel existing orders

Unlike continuous trading, orders are **not matched immediately** when a buy price and sell price are compatible. Instead, all eligible orders remain in the order book until the end of the auction period.

At the end of the auction period, the exchange runs a predefined **auction matching algorithm** to determine the auction price and execute trades.

Example

Assume the following orders are entered during the auction and then create a limit order book:

- Buy 100 shares of BHP at **\$42**
- Sell 100 shares of BHP at **\$39**

Although the buy price is higher than the sell price, these orders **must not trade immediately**. They remain in the order book until the auction matching process begins.

Once the auction ends, the matching engine applies the auction matching algorithm described below to determine how orders are matched and executed.

Orders Matching Logic:

When the auction period ends, the exchange evaluates all active orders from the limit order book and determines a single **matching auction price**.

Orders can be matched if the following conditions are met:

- A buy order can be matched with a sell order when **Buy Price $\geq$ Sell Price**.
- All buy orders with a price **greater than or equal to the auction price** are eligible to participate in matching.
- All sell orders with a price **less than or equal to the auction price** are eligible to participate in matching.

Selecting the Auction Price

Because orders may exist at multiple price levels, there are often several possible auction prices that satisfy the conditions above.

In such cases, the exchange uses a **Maximum Volume Matching** algorithm:

- ➔ Select the auction price that results in the highest total matched quantity.

In other words, the **matching auction price** should maximize the number of shares that can be traded during the auction matching process. Your program is expected to calculate and determine this price.

Example:

Here is the snapshot of the limit order book before auction period ends:

Buy Quantity	Buy Price	Sell Price	Sell Quantity
		102	50000
100	100		
1000	99	99	200
		98	700
500	96		

In this example, prices 98, 99, 100 are potential matching auction prices and you need to find one price (out of these 3) to satisfy the “maximum volume matching algorithm”.

What is expected:

- Good programming practice and algorithm design.
- A reasonable accuracy in the matching logic.
- Bonus: Calculate the total matching volume for the calculated matching auction price.